



סילבוס המחלקה להנדסת תוכנה

Course Title		תכנות מערכות	שם הקורס
Course No.	3503820		מספר קורס
Lecturers' Name	nudler@shenkar.ac.il	יצחק נודלר	מרצה הקורס
			מתרגל הקורס
Year	2025-2026	תשפ"ו	שנת הוראה
Weekly Hours	3		היקף הקורס בש"ש
Credits	3		נקודות זכות
Prerequisite	ארכיטקטורת מחשבים (3502105) רשתות תקשורת (3503836)		דרישות קדם

תקציר הקורס

This course focuses on the GNU/Linux operating system. The students will get familiar with the programming interface and the internals of the operating-system.

- שיעורי הרצאות פרונטליות. - עבודה אינטנסיבית הן במסגרת ההרצאות והן מחוץ להרצאות עם פלטפורמות AI. - שאלות ותשובות בכיתה. - תרגילי בית.	אופני הוראה עיקריים
--	---------------------

תוצאות למידה

A basic understanding of the program development process in a UNIX based system – processes, file I/O, threads, signals, file permissions, etc.

Improved engineering and programming skills.

A deep understanding of the underlying mechanisms of the interface (e.g. system calls) between an application and the kernel.

דרישות מעבר הקורס וחלוקת הציון		
דרישות מעבר הקורס	לפחות 60 במבחן וגם לפחות 60 בציון הסופי המשוקלל	
קטגוריה	% מהציון	הערות
בחינה מסכמת	90%	על <u>כלל</u> החומר שנלמד בקורס.
תרגילי בית	10%	
סה"כ	100%	

נוכחות בקורס	הנוכחות חובה על פי תקנון שנקר
--------------	-------------------------------

מבנה הקורס		
מספר/תאריך	נושא השיעור	פירוט
1	System Programming: תיאוריה, גבולות ומנגנונים	- מה זו מערכת הפעלה? - מה זה ה-kernel של מערכת ההפעלה? - תפקידי ה-kernel - extended/abstract machine - מרכיבי מערכת ההפעלה והקשר הסימביוטי ביניהם - מצבי מעבד (User/Kernel Mode). - user space vs. kernel space - המורכבות הכמותית: LOC והטרוגניות השפות
2	מסלול החיים של המערכת - מתכנון ועד הרצה.	- הפרק הזה הופך את המערכת מ"אוסף רכיבים" (שתארנו בפרק 1) לתהליך הנדסי חי. - אנחנו נתחיל מהנקודה שבה הכל מתחיל - הדרישה למערכת עמידה ואמינה - ונסיים ברגע שבו המשתמש מקליד פקודה ב-Bash.
3	קריאות מערכת - לב ליבו של הקורס	- מה זו קריאת מערכת? - סוגי פסיקות - דגש על פסיקות תוכנה שהן המנגנון העיקרי למימוש קריאות מערכת - טיפול בפסיקות - 'כניסה' ל-kernel דרך/באמצעות ה-wrapper routines שבספרייה glibc
4	תכנות מערכות - על ידי דוגמאות	- תוכנית ה-bash כדוגמא מייצגת של תכנות מערכת ('הפעלה' של ה-kernel של Linux) - 'מאחורי הקלעים' של bash - fork() - exec() - התוכנית cp כדוגמא מייצגת נוספת של תכנות מערכת [open(), read(), write(), close() וכו'] - דוגמא נוספת: my_simple_fork

מבנה מערכת הקבצים. Inodes (Inode Table). Inode ↔ Data Blocks סוגי קבצים. מבנה נתונים: ה-FD כאינדקס ליבת קשר (Context).	File Descriptors, Inodes וארכיטקטורת I/O	5
open(), close() read(), write() טיפול ב-Partial I/O. שינוי מיקום lseek() גישה למטא-דאטה (stat) וקריאה לספריות.	File I/O מעשי	6
Process vs. Program task_dtruct – ייצוג תהליך ב-kernel fork() exec()	תהליכים (Processes)	7
task creation - states and staes transitions - task exit path - orphans ו-zombies -	Tasks Life Cycle	8
Process vs. Thread זיכרון משותף/נפרד pthread_create(), pthread_join :Pthreads	Threads	9
הצורך ב-IPC (isolation) pipe() Named pipe (FIFO) dup2() ו-dup(): I/O redirection	תקשורת בין תהליכים (IPC)	10
משתנים גלובליים (משותפים) מול לוקאליים. Race Condition (הגדרה מדויקת ותנאים). סינכרון באמצעות Mutex: Pthread_mutex_t lock/unlock	Threads Synchronization	11
הגדרה למה משמש מדוע ה-Signals חייבים להיות ממומשים ב-Kernel. סיגנלים משמעותיים, SIGINT, SIGTERM ו-SIGCHLD sigaction(): signal handler	Signals	12
הגדרת סוקט: אוברייקט Kernel. מאפיינים שלסוקט (Port, Type). socket() bind() הסוקט ב-FD: החיבור למודל "הכול קובץ". הבחנה בין סוקט לבין קישור	Sockets: יסודות	13
listen(), accept() טורי: Flow המעבר למקביל: fork() לאחר accept() ניהול סגירת FDs ב-Parent/Child. טיפול ב-Zombies בשרת.	Concurrent client-server	14

רשימה ביבליוגרפית	
W. Richard Setevens, Stephen A. Rago, " Advanced Programming in the UNIX Environment – Second Edition"	
Robert Love, "Linux System Programming"	